LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – INHERITANCE



NAMA: ANDI MUHAMMAD ABID JAYA

NIM: 103132400014

KELAS: S1-SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Teori

Dalam modul ini mempelajari beberapa dasar teori dasar sebagai berikut:

1. Kelas Abstract

Kelas abstract adalah kelas yang dideklarasikan menggunakan keyword abstract dan tidak dapat dibuat menjadi objek secara langsung. Kelas ini dapat memiliki abstract method yang belum memiliki implementasi dan harus diimplementasikan oleh kelas turunannya.

Contohnya, kelas Employee merupakan kelas abstract yang memiliki method earnings() untuk menghitung pendapatan pegawai. Method tersebut diimplementasikan oleh kelas turunannya, seperti SalariedEmployee.

2. Interface

Interface adalah struktur yang mendefinisikan method atau konstanta yang akan digunakan oleh kelas lain. Kelas yang menggunakan interface harus mengimplementasikan method abstrak yang dideklarasikan di dalamnya dengan menggunakan keyword implements.

Contohnya, interface Payable memiliki method getPaymentAmount() yang diimplementasikan oleh kelas Invoice untuk menghitung total pembayaran berdasarkan jumlah barang dan harga per item.

3. Default Method dan Static Method

Default method adalah method pada interface yang memiliki implementasi dan dideklarasikan menggunakan keyword default. Sementara itu, static method merupakan method yang dapat dipanggil melalui nama interface tanpa harus diimplementasikan kembali oleh kelas yang menggunakannya.

Contohnya, interface MyInterface memiliki default method newMethod() dan static method anotherNewMethod() yang dapat digunakan tanpa harus mengimplementasikan kedua method tersebut pada kelas Example.

4. Perbedaan Abstract Class dan Interface

Abstract class dapat memiliki method yang sudah memiliki implementasi, atribut biasa, serta hubungan pewarisan dengan kelas abstract lainnya. Sementara itu, interface digunakan untuk mendefinisikan method dan konstanta yang harus dipenuhi oleh kelas implementasinya.

Contohnya, kelas SalariedEmployee menggunakan keyword extends untuk mewarisi kelas Employee, sedangkan kelas Invoice menggunakan keyword implements untuk menerapkan interface Payable. Interface juga memungkinkan sebuah kelas mengimplementasikan lebih dari satu interface.

B. Guided

1. Guided 1

```
// nama file: A.java

package com.abid.guided1;

public class A {
    int x;
    int y;

    void tampilXY() {
        System.out.println("nilai x: "+x+" nilai y: "+y);
    }
}
```

```
//  nama file: B.java

package com.abid.guided1;

public class B extends A {
    int z;
    int hasil;

    void jumlahXYZ() {
        hasil = (x+y+z);
        System.out.println("jumlah  (x + y + z) = " + hasil );
    }
}
```

```
//  nama file: Demo.java

        package com.abid.guided1;
```

```
public class Demo {
public static void main(String[] args) {

   A superclass=new A();
   B subclass=new B();

   superclass.x = 3;
   superclass.y = 4;

   System.out.println("Superclass: ");
   superclass.tampilXY();

   subclass.x = 1;
   subclass.y = 2;
   subclass.z = 5;

   System.out.println("Subclass:  ");
   subclass.jumlahXYZ();

}
}
```

Output :

```
Superclass:
nilai x: 3 nilai y: 4
Subclass:
jumlah  (x + y + z) = 8
PS D:\SEMESTER 5\PEMROGRAMAN BERBASIS OBJEK\TEMPLATE REPO\praktikum-pbo-ps1ds-05-reg-01>
```

2. Guided 2

```
//   nama file: Employee.java

package com.abid.guided2;

public class Employee {
   private String name;
   String departemen;
```

```
   public Employee(String s) {
      this.name = s;
   }

   public void tampilNama() {
      System.out.println("nama : " + name);
   }
}
```

```
//  nama file: Manager.java

        package com.abid.guided2;

        class Manager extends Employee {
           private String alamat;

           public Manager(String nama, String s) {
           /*memanggil konstruktor employee*/
           super(nama);
           this.alamat = s;
           }

           public void tampilInformasi() {
              super.tampilNama();
              System.out.println("alamat : "  +   alamat);
              System.out.println("departemen : "  + super.departemen);
        }

        }
```

```
//  nama file: Demo.java

        package com.abid.guided2;

        public class Demo {
           public static void main(String[] args) {
              Manager adi = new Manager("adi", "sukabirus");
              adi.departemen = "Personalia";
```

```
            adi.tampilInformasi();
        }
    }
```

Output :

```
nama : adi
alamat : sukabirus
departemen : Personalia
```

3. Guided 3

```
// nama file: Lagu.java

package com.abid.guided3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void CetakKeLayar() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

```
// nama file: Demo.java
package com.abid.guided3;

public class Demo {
    public static void main(String[] args) {
        Lagu d, e;
```

```
        d = new Lagu();
        e = new Lagu();
        d.IsiParam("Lagu 1");
        e.IsiParam("kepastian yang kutunggu", "GiGi");
        d.CetakKeLayar();
        e.CetakKeLayar();
    }
}
```

Output :

```
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : kepastian yang kutunggu, pencipta : GiGi
```

4.

```
// nama File : A.java

        package com.abid.guided4;

        public class A {
            public void tampilkanKeLayar() {
            System.out.println("Method milik class A dipanggil...");
            }
        }
```

```
// nama File : B.java

        package com.abid.guided4;

        public class B extends A {
            public void tampilkanKeLayar() {
            super.tampilkanKeLayar(); // Method milik superclas dipanggil
            System.out.println("Method milik class B dipanggil...");
            }
        }
```

```
// nama File : Demo.java

        package com.abid.guided4;
```

```
public class Main {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output:

```
Method milik class A dipanggil...
Method milik class B dipanggil...
```

C. Unguided

1. Soal Unguided

Soal Unguided — Modul 6 Inheritance

Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Ketentuan

1. Kelas induk (model umum)

- Menyimpan **nama model** (teks) dan **jumlah data latih** (bilangan bulat).
- Memiliki *constructor* yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
- Memiliki method untuk menampilkan informasi.
- Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (*overloading*):
  - Tanpa parameter: biaya = jumlah data latih × 2.
  - Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih × epoch.

2. Kelas turunan pertama (model regresi)

- Mewarisi kelas induk dan tambahkan atribut **jumlah fitur**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah fitur; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).
- Tambahkan **satu method baru** dengan nama yang sama dengan method penghitung biaya, tetapi menerima **dua parameter bilangan bulat** (jumlah *epoch* dan ukuran *batch*): biaya = jumlah data latih × epoch ÷ ukuran batch. Sehingga objek kelas ini memiliki **tiga versi** method penghitung biaya.

3. Kelas turunan kedua (model klasifikasi)

- Mewarisi kelas induk dan menambahkan atribut **jumlah kelas**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah kelas; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
- Versi penghitung biaya dengan parameter *epoch* **tidak** di-override.

4. Kelas turunan ketiga (model klasifikasi *deep learning*)

- Mewarisi **kelas turunan kedua** (bukan kelas induk langsung) dan menambahkan atribut **jumlah layer**.
- *Constructor* menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan *constructor* kelas di atasnya.
- **Override** method: tampilkan informasi dari kelas di atasnya.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

5. Class Main

- Buat tiga objek berikut:

Jenis model	Nama	Data latih	Atribut tambahan
Regresi	`"Regresi Linier"`	500	4 fitur
Klasifikasi	`"Decision Tree"`	800	3 kelas
Klasifikasi *deep learning*	`"Neural Network"`	1000	5 kelas, 3 layer

- Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.
- Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
- Setelah perulangan, tampilkan total biaya.
- Terakhir, tampilkan juga:
  - biaya 10 *epoch* untuk model `"Neural Network"` (ambil dari array),
  - biaya 10 *epoch* untuk model `"Regresi Linier"`,
  - biaya 10 *epoch* dengan ukuran *batch* 50 untuk model `"Regresi Linier"`.

Contoh Output

```
None
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Jawaban :

```
// nama file: Model.java
package com.abid;

public class Model {
    private String nama;
    private int jumlahDataLatih;

    public Model(String nama, int jumlahDataLatih) {
        this.nama = nama;
```

```
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + nama +
                " | Data latih: " + jumlahDataLatih);
    }

    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

```
// nama file: ModelKlasifikasi.java

        package com.abid;

        public class ModelKlasifikasi extends Model {
            private int jumlahKelas;

            public ModelKlasifikasi(String nama, int jumlahDataLatih,
                                    int jumlahKelas) {
                super(nama, jumlahDataLatih);
                this.jumlahKelas = jumlahKelas;
            }

            @Override
            public void tampilkanInfo() {
                super.tampilkanInfo();
                System.out.println("Jumlah kelas: " + jumlahKelas);
            }
```

```
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

```
// nama file: ModelRegresi.java

package com.abid;

public class ModelRegresi extends Model {
    private int jumlahFitur;

    public ModelRegresi(String nama, int jumlahDataLatih,
                        int jumlahFitur) {
        super(nama, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

```
// nama file: ModelDeepLearning.java
```

```
package com.abid;

public class ModelDeepLearning extends ModelKlasifikasi {
   private int jumlahLayer;

   public ModelDeepLearning(String nama, int jumlahDataLatih,
                            int jumlahKelas, int jumlahLayer) {
      super(nama, jumlahDataLatih, jumlahKelas);
      this.jumlahLayer = jumlahLayer;
   }

   @Override
   public void tampilkanInfo() {
      super.tampilkanInfo();
      System.out.println("Jumlah layer: " + jumlahLayer);
   }

   @Override
   public int hitungBiaya() {
      return super.hitungBiaya() + (jumlahLayer * 200);
   }
}
```

```
// nama file: Main.java

package com.abid;

public class Main {
   public static void main(String[] args) {

      ModelRegresi regresi = new ModelRegresi(
            "Regresi Linier", 500, 4);

      ModelKlasifikasi klasifikasi = new ModelKlasifikasi(
            "Decision Tree", 800, 3);

      ModelDeepLearning deepLearning = new ModelDeepLearning(
            "Neural Network", 1000, 5, 3);
```

```
        // Menyimpan objek dalam array bertipe kelas induk
        Model[] daftarModel = {
            regresi,
            klasifikasi,
            deepLearning
        };

        int totalBiaya = 0;

        // Menampilkan informasi dan menghitung total biaya
        for (Model model : daftarModel) {
            model.tampilkanInfo();

            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");

            totalBiaya += biaya;
        }

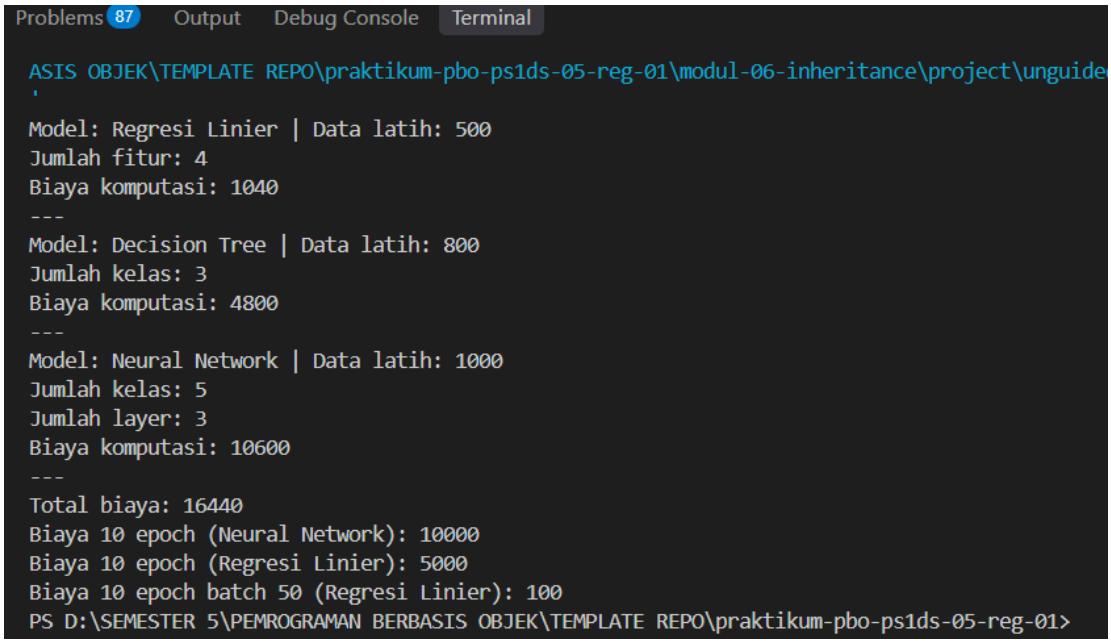
        System.out.println("Total biaya: " + totalBiaya);

        // Biaya 10 epoch untuk Neural Network
        System.out.println("Biaya 10 epoch (Neural Network): "
                + deepLearning.hitungBiaya(10));

        // Biaya 10 epoch untuk Regresi Linier
        System.out.println("Biaya 10 epoch (Regresi Linier): "
                + regresi.hitungBiaya(10));

        // Biaya 10 epoch dengan ukuran batch 50
        System.out.println(
                "Biaya 10 epoch batch 50 (Regresi Linier): "
                + regresi.hitungBiaya(10, 50));
    }
}
```

Output :



Penjelasan :

File Model.java berisi kelas induk Model dengan atribut private nama dan jumlahDataLatih. Constructor digunakan untuk menginisialisasi atribut, sedangkan method tampilkanInfo() menampilkan informasi model. Method hitungBiaya() dan hitungBiaya(int epoch) digunakan untuk menghitung biaya komputasi dengan menerapkan method overloading.

File ModelRegresi.java berisi kelas ModelRegresi yang mewarisi kelas Model dan memiliki atribut private jumlahFitur. Constructor menggunakan super() untuk memanggil constructor kelas induk. Method tampilkanInfo() dan hitungBiaya() di-override, serta method hitungBiaya(int epoch, int ukuranBatch) ditambahkan untuk menghitung biaya berdasarkan epoch dan ukuran batch.

File ModelKlasifikasi.java berisi kelas ModelKlasifikasi yang mewarisi kelas Model dan memiliki atribut private jumlahKelas. Method tampilkanInfo() di-

override untuk menampilkan jumlah kelas, sedangkan hitungBiaya() di-override untuk menghitung biaya komputasi berdasarkan jumlah kelas.

File ModelDeepLearning.java berisi kelas ModelDeepLearning yang mewarisi kelas ModelKlasifikasi dan memiliki atribut private jumlahLayer. Constructor menggunakan super() untuk memanggil constructor kelas induk. Method tampilkanInfo() dan hitungBiaya() di-override untuk menampilkan informasi serta menghitung biaya tambahan berdasarkan jumlah layer.

File Main.java berisi method main() untuk membuat tiga objek model, yaitu Regresi Linier, Decision Tree, dan Neural Network. Ketiga objek disimpan dalam array bertipe Model[] dan diproses menggunakan perulangan untuk menampilkan informasi serta menghitung total biaya. Program juga menghitung biaya berdasarkan epoch dan ukuran batch. Hasil akhirnya adalah total biaya komputasi sebesar 16.440, biaya 10 epoch Neural Network sebesar 10.000, biaya 10 epoch Regresi Linier sebesar 5.000, dan biaya 10 epoch dengan batch 50 sebesar 100.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa kelas abstract digunakan sebagai kelas dasar yang tidak dapat diinstansiasi secara langsung dan dapat memiliki abstract method yang harus diimplementasikan oleh kelas turunannya. Praktikum ini juga menunjukkan penerapan interface sebagai kontrak yang mendefinisikan method untuk diimplementasikan oleh kelas lain menggunakan keyword implements. Selain itu, constructor dan pewarisan digunakan untuk menginisialisasi atribut serta menghubungkan kelas dengan kelas induknya.

Praktikum ini juga membahas penggunaan default method dan static method pada interface yang memungkinkan method memiliki implementasi tanpa harus ditulis ulang pada kelas implementasi. Perbedaan antara abstract class dan interface terlihat pada implementasi method, atribut, serta mekanisme pewarisannya. Dengan demikian,

praktikum ini membantu memahami konsep abstract class dan interface serta penerapannya dalam pemrograman berorientasi objek menggunakan Java.